AF 52

Synopsis

AI Resume Analyser and Job Recommendation Portal

Submitted in partial fulfilment of the requirements of the degree of

Bachelor of Technology

In

Electronic and Computer Science

by

Kunal Bhandarkar [EU1237030]

Rishi Jadhav [EU1237028]

Nishit Manjrekar [EU1237038]

Harsh Patil [EU2237012]

Under the Guidance of

Ms. Aishwarya Churi

Project guide

DEPARTMENT OF ELECTRONICS AND COMPUTER SCIENCE

ST. JOHN COLLEGE OF ENGINEERING & MANAGEMENT

UNIVERSITY OF MUMBAI

2026-2027

Certificate of Approval

This is to certify that Kunal Bhandarkar (EU1237030), Rishi Jadhav (EU1237028), Nishit Manjrekar (EU1237038), Harsh Patil (EU2237012), are bonafide students of Electronic and Computer Science Department, St. John College of Engineering and Management, Mumbai. They have satisfactorily completed the requirements of Synopsis as prescribed by **ST. JOHN COLLEGE OF ENGINEERING AND MANAGEMENT (An Autonomous College affiliated to University of Mumbai),** while working on "AI Resume Analyser and Job Recommendation Portal".

Signature : .....
Name : Ms. Aishwarya Churi
Project guide,

Signature : .....
Name : Dr.Md.Imaduddin
HOD

Signature: .....
Name : Dr. Kamal Shah
Principal,
St. John College of Engineering and Management.

Declaration

I/we declare that this written submission represents my/our ideas in my/our own words and where others ideas or words have been included, I/we have adequately cited and referenced the original sources. I/we also declare that I/we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my/our submission. I/we understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

.....
(Kunal Bhandarkar – EU1237030)

.....
(Rishi Jadhav – EU1237028)

.....
(Nishit Manjrekar – EU1237038)

.....
(Harsh Patil – EU2237012)

Date:

Acknowledgement

We express our sincere gratitude to all those who have directly and indirectly contributed to the successful preparation of this project synopsis.

We are deeply grateful to **Dr. Kamal Shah, Principal, St. John College of Engineering and Management**, for providing an excellent academic environment and the necessary facilities to undertake this project.

We We extend our heartfelt thanks to the Head of the Department of Electronics and Computer Science for their constant encouragement, valuable suggestions, and continuous support throughout the preparation of this synopsis. heir con this syn

We would like to express our sincere appreciation to our project guide, **Dr./Prof.** _____, for their invaluable guidance, constructive suggestions, technical expertise, and continuous motivation. Their encouragement and insightful feedback have been instrumental in shaping this project synopsis.

We also acknowledge all the faculty members and laboratory staff of the Department of Computer Engineering for their timely assistance, cooperation, and technical support.

We also thank the author of the open-source Smart AI Resume Analyzer project, whose public documentation served as a practical reference for this work.

Finally, we express our heartfelt gratitude to our parents, family members, friends, and classmates for their constant encouragement, moral support, and cooperation throughout this work.

Group Members

1. Kunal Bhandarkar [EU1237030]
2. Rishi Jadhav [EU1237028]
3. Nishit Manjrekar [EU1237038]
4. Harsh Patil [EU2237012]

ABSTRACT

Applicant Tracking Systems (ATS) are widely used to screen applications before a human reads them, yet most students and early-career professionals receive little feedback on how their resumes are interpreted by such systems. Existing resume checkers often rely on simple keyword counts, give unexplained scores, and treat resume improvement and job search as separate activities.

This synopsis proposes an **AI Resume Analyser and Job Recommendation Portal**, a web application in which a user uploads a PDF or DOCX resume and receives an itemised resume score, keyword and skill-gap analysis, AI-generated improvement suggestions and a ranked list of suitable jobs with explanations. The proposed methodology combines a rule-based ATS-compatibility check, TF-IDF keyword coverage, Sentence-BERT semantic similarity and a large language model that converts the computed gaps into readable advice. Job recommendation is content-based: the candidate profile and job postings are embedded as vectors and ranked by cosine similarity combined with explicit skill overlap, and each recommendation lists matched and missing skills together with suggested courses.

The design also treats privacy and fairness as requirements: personal identifiers are masked before any external API call, user consent and data-retention limits are built in, credentials are stored as salted hashes, and attributes such as name, gender and photograph are excluded from scoring. The system will be implemented in Python using Streamlit, spaCy, scikit-learn, sentence-transformers and SQLite, and will be evaluated for parsing accuracy, score validity, recommendation quality, response time and usability.

The work takes an existing open-source project as a practical reference and proposes improvements in semantic matching, explainability, privacy and security. The expected outcome is a deployable prototype, an evaluation dataset and a documented methodology that can help students, placement cells and recruiters.

Keywords: *resume analysis, applicant tracking system, natural language processing, Sentence-BERT, job recommendation, skill gap, explainable AI.*

C O N T E N T S

- List of Figures vii
- List of Tables vii
- Abbreviations and symbols viii

Chapter No.	Topic	Page No.
Chapter 1	Introduction: Background, Problem Statement, Need, Objectives, Scope, Expected Outcomes	1
Chapter 2	Literature Review: Survey table, Research Gap, Novelty/Innovation	4
Chapter 3	Proposed Methodology: Working Principle, Architecture/Block Diagram, Flowchart, Methodology, Components	8
Chapter 4	Project Planning: WBS, Gantt Chart, Roles, Risk Assessment	18
Chapter 5	Resources & Feasibility: Technical/Economic/Operational Feasibility, Budget, BOM	21
Chapter 6	Expected Results: Prototype, Accuracy, Efficiency, Performance, Experimental results	23
Chapter 7	Sustainability & Impact: Industrial relevance, Societal and Environmental impact, SDG mapping, Future Scope	25
Chapter 8	Conclusion	27
References	IEEE style references	28
Appendices	Requirements, test cases, questionnaire, LLM prompt, skill aliases, project canvas	30

List of Figures

Figure No.	Figure Name	Page No.
3.1	Resume-processing pipeline	9
3.2	System architecture / Block diagram	9
3.3	Data Flow Diagram (DFD), level 1	10
3.4	Flow chart of resume analysis and job recommendation	11
3.5	Entity–Relationship diagram	13
3.6	UML use-case diagram	14
3.7	GUI 01 – Resume upload and analysis request	15
3.8	GUI 02 – Analysis dashboard	15
3.9	GUI 03 – Job recommendations with explanations	16
4.1	Gantt chart for semester planning	18

List of Tables

Table No.	Table Name	Page No.
2.1	Survey of recent and foundational work	4
2.2	Feature comparison	5
3.1	Components of the resume score	12
3.2	Software components and specifications	17
4.1	Work breakdown structure and planning table	18
4.2	Roles and responsibilities	19
4.3	Risk assessment	19
4.4	Project milestones	20
5.1	Estimated budget (INR)	22
5.2	Bill of materials	22
6.1	Expected performance targets	24
6.2	Planned configurations for ablation study	24
6.3	Objectives and expected evidence	24
7.1	Mapping of the project to the SDGs	26
A.1	Sample test cases	31
A.2	Examples of skill normalisation	32

Abbreviations and Symbols

Abbreviation	Full form
AI	Artificial Intelligence
ATS	Applicant Tracking System
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BOM	Bill of Materials
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DOCX	Office Open XML Word Document
ER	Entity–Relationship
GUI	Graphical User Interface
HR	Human Resources
LLM	Large Language Model
MRR	Mean Reciprocal Rank
ML	Machine Learning
NDCG	Normalised Discounted Cumulative Gain
NER	Named Entity Recognition
NLP	Natural Language Processing
OCR	Optical Character Recognition
PDF	Portable Document Format
PII	Personally Identifiable Information
SBERT	Sentence-BERT
SDG	Sustainable Development Goal
SUS	System Usability Scale
TF-IDF	Term Frequency–Inverse Document Frequency
UI	User Interface
UML	Unified Modeling Language
WBS	Work Breakdown Structure

CHAPTER 1: INTRODUCTION

1.1 Background

Searching for a job has become a digital, data-driven process. A single opening advertised online can attract hundreds of applications within days, and recruiters rarely have the time to read every resume in full. To cope with this volume, most medium and large organisations route applications through an Applicant Tracking System (ATS), a software platform that parses each resume, stores it in a structured form and ranks it against the requirements of the vacancy. A resume that is well written but poorly structured, or that uses different vocabulary from the job description, may therefore never reach a human reader.

At the same time, advances in Natural Language Processing (NLP) have changed what software can do with text. Statistical methods such as Term Frequency–Inverse Document Frequency (TF-IDF) made it possible to measure how strongly a document is associated with particular terms [3]. Neural language models such as BERT and Sentence-BERT now represent whole sentences as dense vectors, so that two texts with the same meaning but different wording lie close together in vector space [1], [2]. Large Language Models (LLMs) can additionally read a resume and produce human-readable advice in plain language [7], [26].

Students and early-career professionals are affected most by this change. They typically have limited exposure to how recruiters and ATS software read a resume, they rarely receive personalised feedback, and they often do not know which skills the market currently expects for a given role. A tool that analyses a resume, explains its weaknesses and then recommends suitable jobs and learning resources can therefore act as a low-cost career-guidance assistant. This project proposes the design and development of such a tool: an **AI Resume Analyser and Job Recommendation Portal**.

1.2 Problem Statement

Job seekers, particularly fresh graduates, lack a single, trustworthy, explainable tool that tells them how their resume will be read by automated screening systems and which openings they are genuinely qualified for. The problem can be described under four heads.

- **Existing problem:** Many free resume checkers return only a numerical score or a generic checklist. They rely mainly on exact keyword matching, so a candidate who writes “data visualisation” is penalised when the job description says “dashboards”. Job search and resume improvement are also handled in separate tools, forcing the candidate to move between platforms.
- **Industrial issue:** Recruiters receive far more applications than they can review, and ATS filters reject resumes for avoidable reasons such as multi-column layouts, missing section headings or

absent keywords. Organisations lose qualified candidates, while candidates lose opportunities for reasons unrelated to their ability.

- **Research gap:** Published work on person–job fit concentrates on large proprietary datasets and deep neural models [8]–[11]. There is little work on a lightweight, open, student-deployable system that combines rule-based ATS checks, semantic matching and LLM feedback in one explainable pipeline, and that also treats privacy and bias as design requirements [14], [15].
- **Need for improvement:** Existing open-source projects, including the reference project examined in this synopsis, provide resume analysis and job listings but do not document semantic job matching, per-recommendation explanations, personal-data masking before external API calls, or hashed credential storage. These are the improvements the proposed system targets.

1.3 Need of the Project

The project is important for three groups of stakeholders. **Job seekers** need objective, actionable feedback delivered within seconds, and a clear view of the skills they must acquire for a target role. **Training and placement cells** in colleges need a tool that can help large batches of students prepare resumes of a consistent quality before campus drives. **Recruiters** benefit indirectly because resumes become better structured and better matched to the roles to which they are submitted.

There is also an academic need. A final-year engineering project in this area exercises several core competencies at once: text extraction, NLP, information retrieval, recommender-system design, database design, web development, secure coding and software testing. It provides a realistic setting in which students must also reason about ethical issues such as fairness, consent and data protection under the Digital Personal Data Protection Act, 2023 [20].

1.4 Objectives

General objective

To design, develop and evaluate a web-based AI system that analyses a candidate's resume against industry expectations and a target job description, explains the result in plain language, and recommends relevant job openings and learning resources.

Specific objectives

1. Design a resume-parsing pipeline that extracts text from PDF and DOCX files and identifies sections, skills, education, experience and projects.
2. Develop a hybrid scoring engine that combines ATS-format rules, TF-IDF keyword coverage and Sentence-BERT semantic similarity into an interpretable resume score.

3. Develop a job-recommendation module that ranks job postings by similarity to the candidate profile and explains each recommendation through matched and missing skills.
4. Integrate an LLM service to generate section-wise improvement suggestions, while masking personal identifiers before any external call.
5. Implement a secure web portal with user authentication, resume builder, PDF report export, feedback collection and an administrator dashboard.
6. Test the system for functional correctness, accuracy, response time and usability, and document the results.

1.5 Scope

What will be covered

- Resume upload (PDF, DOCX), text extraction and structured parsing for English-language resumes.
- ATS-compatibility check, keyword-gap analysis, semantic similarity against a pasted job description, and role-based skill-gap analysis for a defined set of technology and business roles.
- Job recommendation from a curated job-posting store populated through licensed or public job-listing sources, with course suggestions mapped to missing skills.
- Resume builder with multiple templates, PDF report generation, user and administrator modules.

Limitations

- The first release supports English resumes only and a limited number of job families (for example software, data, and business roles).
- Image-only (scanned) resumes require OCR and will be handled only as a best-effort feature.
- LLM output is advisory; the system will not make hiring decisions, and recommendations depend on the quality and freshness of the job data available.

Future expansion

Regional-language support, mock-interview preparation, employer-side candidate shortlisting with fairness audits, and a mobile application are identified as extensions in Section 7.5.

1.6 Expected Outcomes

- A working web application (software prototype) hosted on a cloud platform, with source code maintained in a version-controlled repository.
- A resume-analysis module with documented scoring methodology and a job-recommendation module with explainable output.
- A labelled evaluation set of consenting, anonymised resumes and job descriptions, together with test reports.

- A project report, user manual and a possible research paper or conference publication on hybrid explainable resume–job matching.
- A potential start-up or campus-placement tool, since the architecture can be adapted by training and placement cells.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter surveys the research and software that inform the proposed system. The literature falls into four groups: text representation (TF-IDF, word and sentence embeddings, transformers); resume parsing and skill extraction; person–job fit and job recommendation; and responsible use of AI in hiring. The final entry in the survey is the open-source project that served as the practical reference for this synopsis. Its documentation (the public README of the repository) describes a Streamlit application with resume analysis, a resume builder, an administrator dashboard and a job-search feature; it was reviewed as a software reference only, and no code is claimed as original work of this team.

2.2 Literature Review Table

Table 2.1: Survey of recent and foundational work

Paper	Author	Year	Method	Findings	Research Gap
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [1]	J. Devlin, M. Chang, K. Lee, K. Toutanova	2019	Transformer encoder pre-trained with masked-language and next-sentence objectives	Produces context-aware text representations that can be fine-tuned for many NLP tasks	General-purpose; not tailored to resumes or job matching without further work
Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks [2]	N. Reimers, I. Gurevych	2019	Siamese/triplet BERT networks that output fixed-size sentence vectors	Semantic similarity can be computed efficiently with cosine distance over embeddings	Evaluated on generic similarity benchmarks, not on resume–job pairs
Term-weighting approaches in automatic text retrieval [3]	G. Salton, C. Buckley	1988	TF-IDF and related term-weighting schemes for vector-space retrieval	Weighting terms by frequency and rarity improves retrieval effectiveness	Exact-term matching; cannot capture synonyms or paraphrases
End-to-End Resume Parsing and Finding Candidates for a Job Description using BERT [4]	V. Bhatia, P. Rawat, A. Kumar, R. R. Shah	2019	BERT-based entity extraction from resumes followed by candidate–job matching	Neural models can parse unstructured resumes and support matching to a job description	Focus on recruiter-side ranking; no candidate-side feedback or recommendations
Matching Resumes to Jobs via Deep Siamese Network [6]	S. Maheshwary, H. Misra	2018	Siamese deep network that embeds resumes and job descriptions in a shared space	Learned similarity can outperform simple keyword overlap for resume–job matching	Needs large labelled data; limited explanation of why a pair matches
Enhancing Person-Job Fit for Talent Recruitment: An Ability-aware Neural Network Approach [8]	C. Qin et al.	2018	Recurrent network with hierarchical attention over ability requirements and experiences	Modelling individual abilities improves person–job fit prediction	Relies on proprietary recruitment logs; not reproducible by students
Person-Job Fit: Adapting the Right Talent for the Right Job with Joint Representation Learning [9]	C. Zhu et al.	2018	CNN-based joint representation of job postings and resumes	Joint learning captures semantic compatibility between job and candidate text	Black-box scores; no skill-gap or learning guidance for the candidate

Paper	Author	Year	Method	Findings	Research Gap
SKILL: A System for Skill Identification and Normalization [10]	M. Zhao et al.	2015	Skill extraction and normalisation against a taxonomy at industrial scale	Normalising skill mentions to canonical entries improves downstream matching	Proprietary taxonomy and infrastructure; not available as an open component
Personalized Job Recommendation System at LinkedIn: Practical Challenges and Lessons Learned [11]	K. Kenthapadi, B. Le, G. Venkataraman	2017	Production job-recommendation pipeline using candidate, job and interaction features	Highlights practical issues: cold start, data quality, scale and evaluation	Large-scale industrial design; not a template for small, self-hosted systems
Content-based Recommender Systems: State of the Art and Trends [12]	P. Lops, M. de Gemmis, G. Semeraro	2011	Survey of content-based recommendation using item and profile representations	Content-based methods suit domains with rich text and mitigate cold start for new items	Survey; no resume or job domain implementation
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [5]	P. Lewis et al.	2020	Combines a retriever with a generative language model	Grounding generation in retrieved text improves factual accuracy	Not applied to resume feedback; grounding of career advice remains open
Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices [14]	M. Raghavan, S. Barocas, J. Kleinberg, K. Levy	2020	Review of vendor claims and practices in algorithmic hiring tools	Many tools give little evidence of validity or bias testing	Offers analysis rather than a concrete, auditable design for small systems
Smart AI Resume Analyzer (open-source project) [16]	H. Patel (Hunterdii)	2025	Streamlit app using spaCy, scikit-learn, Gemini API, SQLite; job search via web scraping	Offers ATS score, keyword and skill gaps, resume builder, dashboard and job search	Semantic matching, explainable recommendations, PII masking and hashed credentials not documented

Table 2.2 compares the capabilities of typical keyword-based resume checkers, the reference open-source project (as documented in its README) and the proposed system. “Not documented” means that the feature is not described in the public documentation, not that it is proven absent.

Table 2.2: Feature comparison

Feature	Typical keyword checkers	Reference project [16]	Proposed system
ATS-format rules	Basic checklist	ATS compatibility score	Rule-based checks with itemised pass/fail
Keyword gap	Exact match	Keyword gap analysis	TF-IDF weighted, with synonym/alias normalisation
Semantic matching	No	Not documented	Sentence-BERT cosine similarity
LLM feedback	Rare	Gemini-based analysis	LLM feedback on masked text, grounded in computed gaps
Job recommendation	No	Job search (scraping)	Embedding-based ranking from licensed/open sources
Explainability	Low	Not documented	Matched/missing skills shown for every job
Privacy and bias controls	Rarely stated	Not documented	PII masking, consent, retention limits

Feature	Typical keyword checkers	Reference project [16]	Proposed system
Credential security	Varies	Default admin credentials published	Salted password hashing, role-based access

2.3 Research Gap

What previous researchers have done

Earlier work has shown that term weighting, learned embeddings and neural networks can all be used to compare resumes with job descriptions [3], [6], [8], [9]. BERT-style models have been used to parse resumes and match candidates [4], and industrial systems have documented the practical challenges of recommending jobs at scale [11]. Skill taxonomies and normalisation have been shown to improve matching quality [10]. Reviews of hiring technology have also warned that validity and fairness claims are often unsupported [14].

What remains unsolved

Most published systems are recruiter-facing, trained on proprietary data and reported as black-box scores. Candidate-facing tools available to students are generally limited to keyword counting, or they offer LLM-generated advice without grounding in measurable gaps. Few open implementations connect resume analysis with job recommendation in one flow, explain why each job is recommended, or treat privacy-preserving design (such as masking names and contact details before calling an external API) as a core requirement.

How this project addresses the gap

The proposed portal integrates rule-based, statistical and neural techniques in a transparent pipeline. Each score is decomposed into components the user can inspect; each recommended job lists matched and missing skills; LLM prompts are built from computed gaps rather than from raw text alone; and personal identifiers are removed before external calls. The system is designed to run on commodity hardware using open-source libraries so that it can be reproduced and extended by other students and institutions.

2.4 Novelty / Innovation

Innovative idea

A single, explainable pipeline that moves the candidate from **"how good is my resume?"** to **"which jobs fit me, and what should I learn next?"**, using the same candidate profile at every step.

Unique features

- Hybrid scoring: ATS rules + TF-IDF coverage + Sentence-BERT similarity + content-quality checks, with published weights.

- Alias-based skill normalisation (for example “JS” and “JavaScript”, “ML” and “Machine Learning”) so that equivalent skills are counted once.
- Explainable recommendations: every job card lists matched skills, missing skills and a similarity score.
- Privacy-by-design: name, e-mail, phone and address are masked before any LLM call; explicit consent and a retention policy are built into the workflow.
- Skill-gap to learning-path mapping, linking each missing skill to curated courses.

Improvements over existing systems

Compared with keyword-only checkers, the system understands paraphrases through embeddings. Compared with the reference project, it proposes licensed or public job sources in place of scraping, salted password hashing in place of documented default credentials, and explainability and bias-mitigation features that are not described in the reference documentation.

2.5 Summary of the Survey and Design Implications

The survey leads to five design decisions that shape the methodology in Chapter 3:

1. **Combine lexical and semantic matching.** TF-IDF is transparent and fast [3], while sentence embeddings capture paraphrases [2]; using both gives robustness and interpretability.
2. **Normalise skills.** Matching quality depends on mapping different surface forms to canonical skills [10], so an alias table is a core component.
3. **Prefer content-based recommendation.** With no interaction history for new users, content-based ranking is appropriate and avoids the cold-start problem [11], [12].
4. **Ground LLM output.** Generated advice should be built from measured gaps and shown with the underlying evidence, in the spirit of retrieval-grounded generation [5].
5. **Design for accountability.** Because hiring tools are criticised for weak validity and possible bias [14], [15], the system exposes component scores, avoids sensitive attributes and limits data retention.

CHAPTER 3: PROPOSED METHODOLOGY

3.1 Proposed System

The proposed system is a web portal in which a registered user uploads a resume, optionally selects a target role or pastes a job description, and receives (i) a resume score with an itemised breakdown, (ii) AI-generated improvement suggestions, (iii) a ranked list of matching jobs with explanations, and (iv) learning resources for missing skills. The user can also create a new resume with a template-based builder and download a PDF report. An administrator can view usage statistics, manage users and review feedback.

The system is organised into seven cooperating modules:

- **Authentication and user management:** registration, login, role-based access (user, administrator), salted password hashing.
- **Resume parser:** text extraction, cleaning, section detection, named-entity recognition and skill extraction.
- **Scoring engine:** ATS rules, TF-IDF keyword coverage, semantic similarity, completeness and quality checks.
- **LLM advisor:** builds a structured prompt from the masked profile and the computed gaps, and returns section-wise suggestions.
- **Job recommender:** embeds job postings and the candidate profile, ranks jobs and generates explanations and course links.
- **Resume builder and report generator:** templates, live preview and PDF export.
- **Analytics and feedback:** dashboards for users and administrators, feedback form, usage logs.

3.2 Working Principle

The working of the system can be described step by step:

1. The user registers, logs in and gives consent for processing of resume data.
2. The user uploads a PDF or DOCX resume; the system validates file type and size.
3. Text is extracted and cleaned; headings such as Education, Skills, Experience and Projects are detected to split the resume into sections.
4. spaCy NER and pattern rules [21] identify entities; contact details are separated and masked; skills are matched to a normalised skill dictionary.
5. The scoring engine computes component scores and a weighted overall score; keyword and skill gaps are computed for the chosen role or job description.

6. A prompt built from the masked profile and the computed gaps is sent to the LLM, which returns suggestions in a fixed JSON schema.
7. The candidate profile is embedded and compared by cosine similarity with stored job-posting embeddings; the top-N jobs are returned with explanations and course suggestions.
8. Results are displayed on the dashboard, stored in the database and made available as a downloadable PDF report. The user may edit the resume and re-run the analysis.

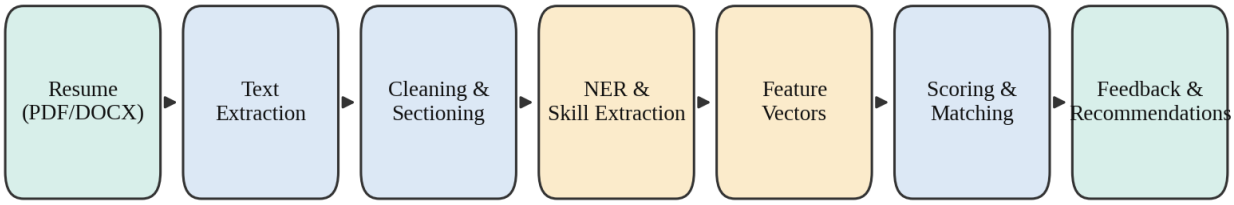


Fig. 3.1: Resume-processing pipeline

3.3 System Architecture / Block Diagram

The system follows a four-layer architecture (Fig. 3.2). The presentation layer is a Streamlit interface; the application layer contains the business logic as Python modules; the intelligence layer wraps the NLP, embedding and LLM components; and the data layer stores users, resumes, job postings, embeddings and logs. Layers communicate only through well-defined function interfaces so that, for example, the embedding model or the LLM provider can be replaced without changing the user interface.

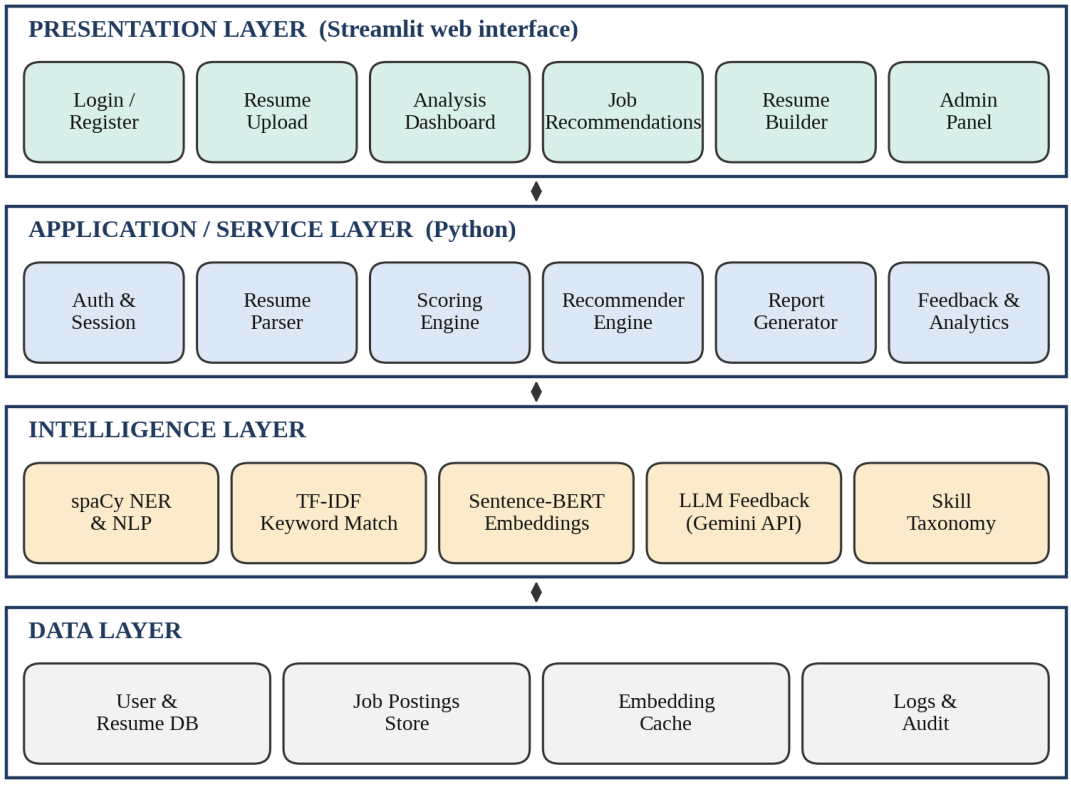


Fig. 3.2: System architecture / block diagram

The data flow between users, processes, data stores and external services is shown in the level-1 Data Flow Diagram (Fig. 3.3). External services are limited to the LLM API and the job-data source; both receive only the minimum data required.

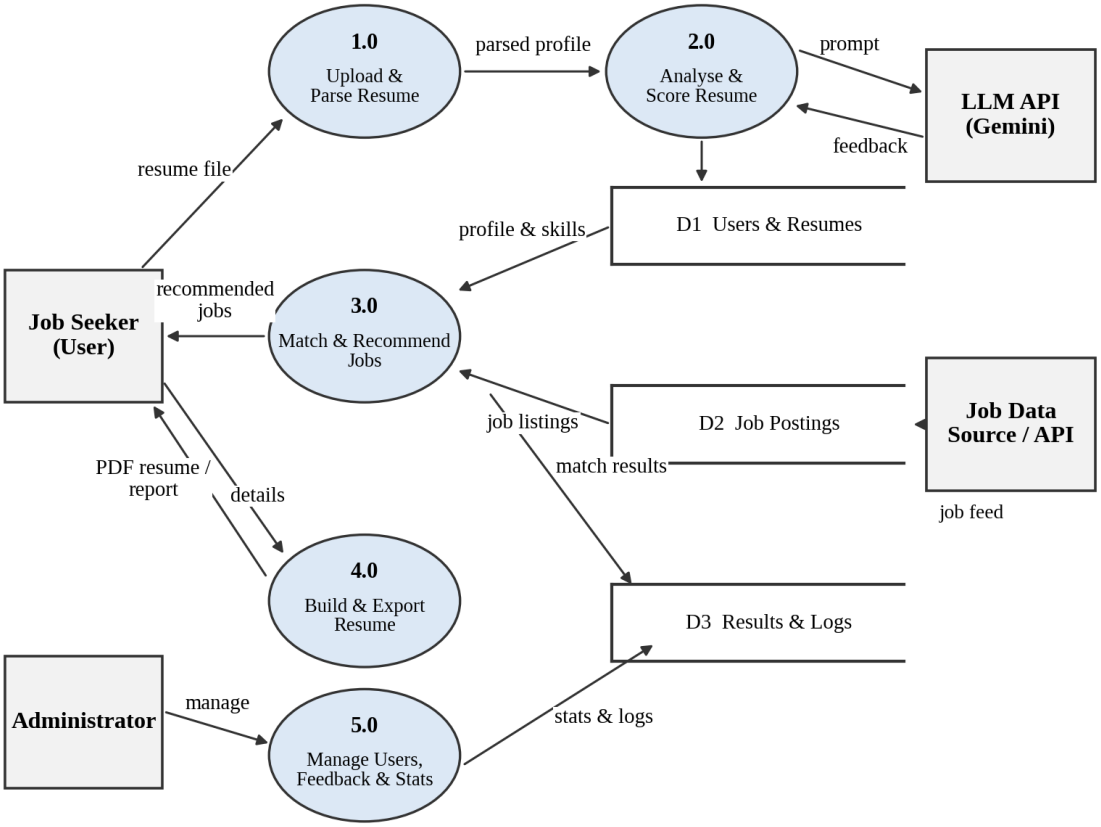


Fig. 3.3: Data Flow Diagram (DFD), level 1

3.4 Flow Chart

Fig. 3.4 shows the end-to-end control flow for a single analysis session, including the validation branch for unsupported files and the iterative loop in which the candidate improves the resume and re-submits it.

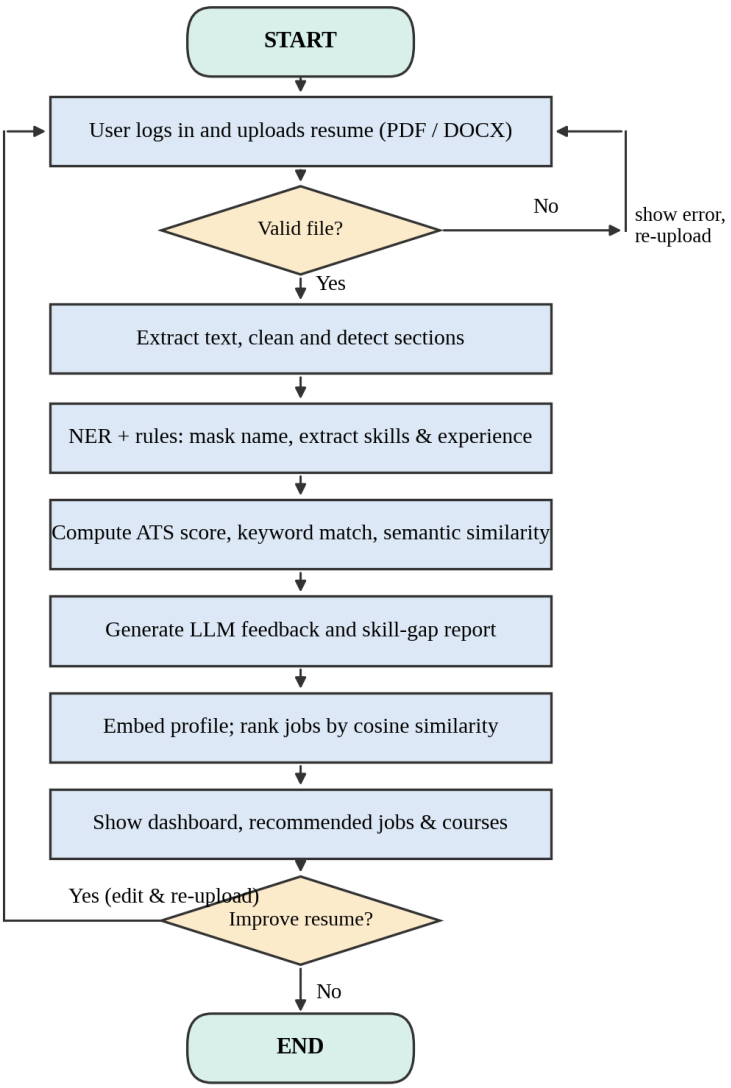


Fig. 3.4: Flow chart of resume analysis and job recommendation

3.5 Methodology

3.5.1 Design procedure

An iterative, agile approach with two-week sprints will be followed [28]. Requirements are captured as user stories (Appendix A), converted into UML diagrams, and implemented module by module with unit tests. Each sprint ends with a demonstration to the guide and a review of the backlog. Design documents follow the recommendations of IEEE Std 830 and ISO/IEC 25010 for requirement specification and software quality [18], [19].

3.5.2 Resume parsing and skill extraction

Text is extracted page by page and normalised (Unicode clean-up, removal of repeated whitespace and page headers). Section boundaries are found with a heading dictionary and layout hints. Skills are identified by phrase matching against a curated skill list, extended by spaCy noun-phrase and NER output,

and then mapped to canonical names through an alias table. Experience duration is estimated from date ranges.

3.5.3 Scoring algorithm

The overall resume score S is a weighted sum of five component scores, each scaled to the range 0 to 100 (Table 3.1). The weights are initial design values and will be tuned during validation against expert ratings.

$$S = 0.20 \cdot F + 0.25 \cdot K + 0.25 \cdot M + 0.15 \cdot C + 0.15 \cdot Q$$

Table 3.1: Components of the resume score

Component	How it is computed	Weight
ATS format compliance	Single-column layout, standard headings, parsable text, file type, length	20 %
Keyword coverage	TF-IDF weighted overlap between resume and role/job keywords	25 %
Semantic similarity	Cosine similarity of Sentence-BERT embeddings (resume vs. job description)	25 %
Section completeness	Presence of contact, summary, skills, education, experience, projects	15 %
Content quality	Action verbs, quantified achievements, bullet length, repetition	15 %

The keyword score uses TF-IDF (implemented with scikit-learn [17]), where the weight of a term t in a document d is the product of its term frequency and its inverse document frequency [3]:

$$tfidf(t, d) = tf(t, d) \times \log(N / df(t))$$

The semantic score uses the cosine similarity between the embedding vectors of the resume (a) and the job description (b) [2]:

$$\cos(a, b) = (a \cdot b) / (||a|| \cdot ||b||)$$

3.5.4 Job-recommendation algorithm

Each job posting is converted to an embedding once, when it is added to the store, and cached. For a candidate, the profile embedding is compared with all job embeddings. The final match score combines semantic similarity with explicit skill overlap and a small fit term for experience level and location preference:

$$Match(r, j) = 0.60 \cdot \cos(e_r, e_j) + 0.30 \cdot |Skills_r \cap Skills_j| / |Skills_j| + 0.10 \cdot Fit(r, j)$$

The top-N jobs are returned together with the matched skills, the missing skills and suggested courses for each missing skill. The approach is content-based, which avoids the cold-start problem for new users and new jobs [12], [13]; interaction data from saved and dismissed jobs can later be used to re-rank results.

```
function RECOMMEND(resume_text, role, N):
    profile  <- MASK_PII(PARSE(resume_text))
    e_r      <- EMBED(profile.summary + profile.skills)
    for each job j in JOB_STORE:
        sem   <- COSINE(e_r, j.embedding)
        ovl   <- |profile.skills ∩ j.skills| / |j.skills|
        fit   <- EXPERIENCE_FIT(profile, j) + LOCATION_FIT(profile, j)
        score <- 0.60*sem + 0.30*ovl + 0.10*fit
        j.matched <- profile.skills ∩ j.skills
        j.missing <- j.skills − profile.skills
    return TOP_N(JOB_STORE, by=score, n=N) with COURSES(j.missing)
```

3.5.5 LLM feedback and privacy controls

The LLM is used only to turn computed findings into readable advice. The prompt contains the masked profile, the list of missing keywords and the lowest-scoring components, and asks for a fixed JSON schema (strengths, weaknesses, rewritten bullet examples). Contact details and the candidate's name are removed before the call, API keys are read from environment variables, and responses are validated before display. Because LLM output can be inaccurate, the interface labels it as advisory and shows the underlying numeric evidence beside it.

3.5.6 Database design

A relational schema (Fig. 3.5) stores users, resumes, analyses, job postings, recommendations and feedback. Passwords are stored only as salted bcrypt hashes. Uploaded files and parsed text are deleted on user request and automatically after a configurable retention period.

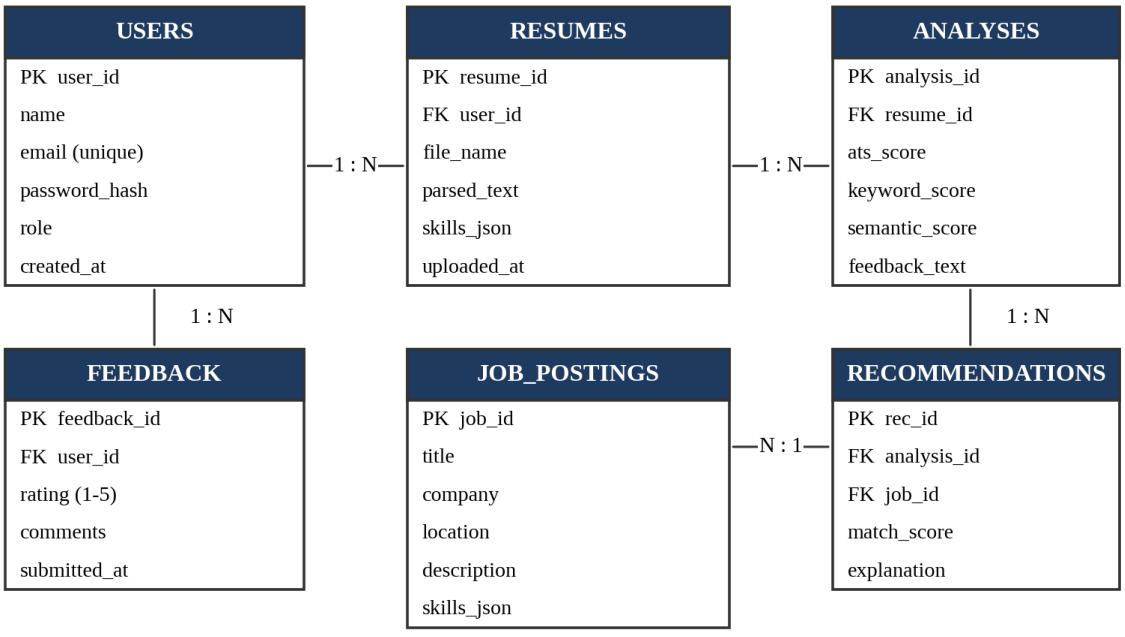


Fig. 3.5: Entity–Relationship diagram

3.5.7 UML and user interface design

The use-case diagram (Fig. 3.6) identifies three actors: the job seeker, the administrator and the external LLM service. Three representative screens of the planned interface are shown as wireframes in Fig. 3.7 to Fig. 3.9; they are design sketches and not screenshots of a finished product.

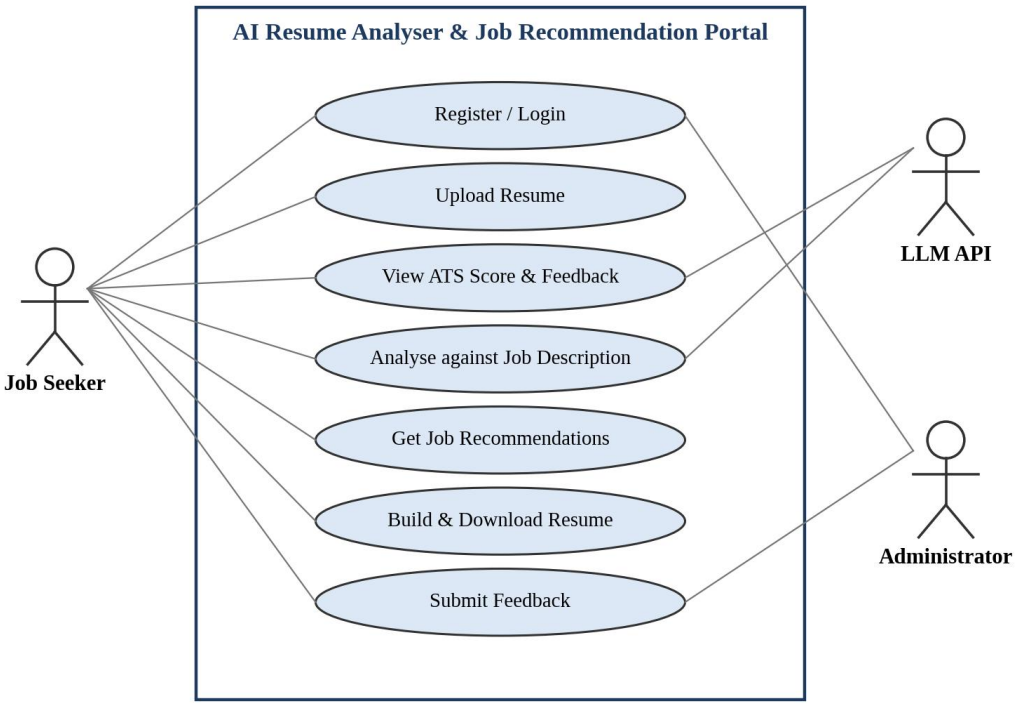


Fig. 3.6: UML use-case diagram

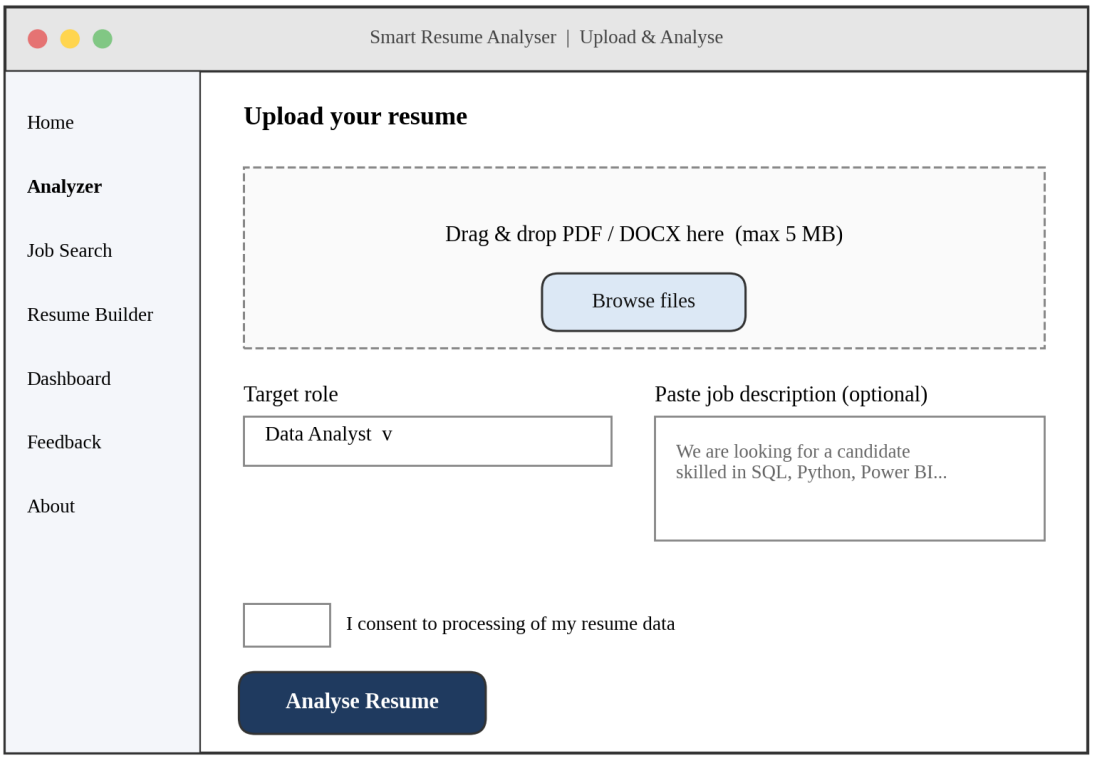


Fig. 3.7: GUI 01 – Resume upload and analysis request

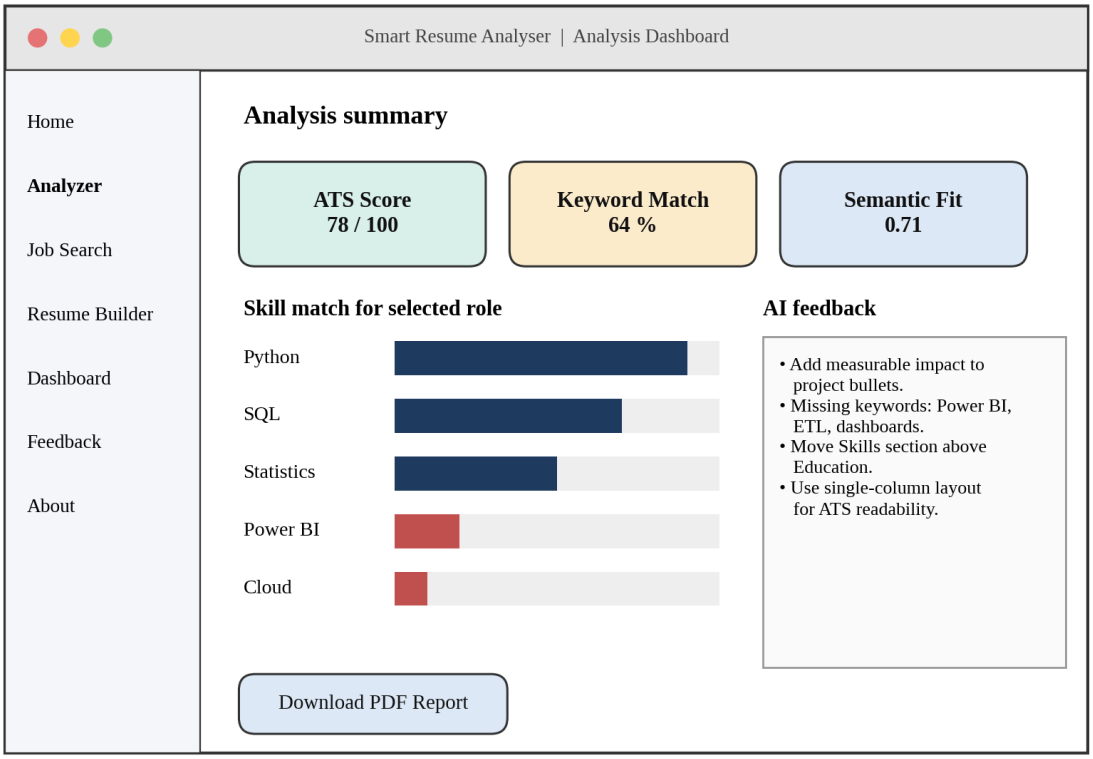


Fig. 3.8: GUI 02 – Analysis dashboard

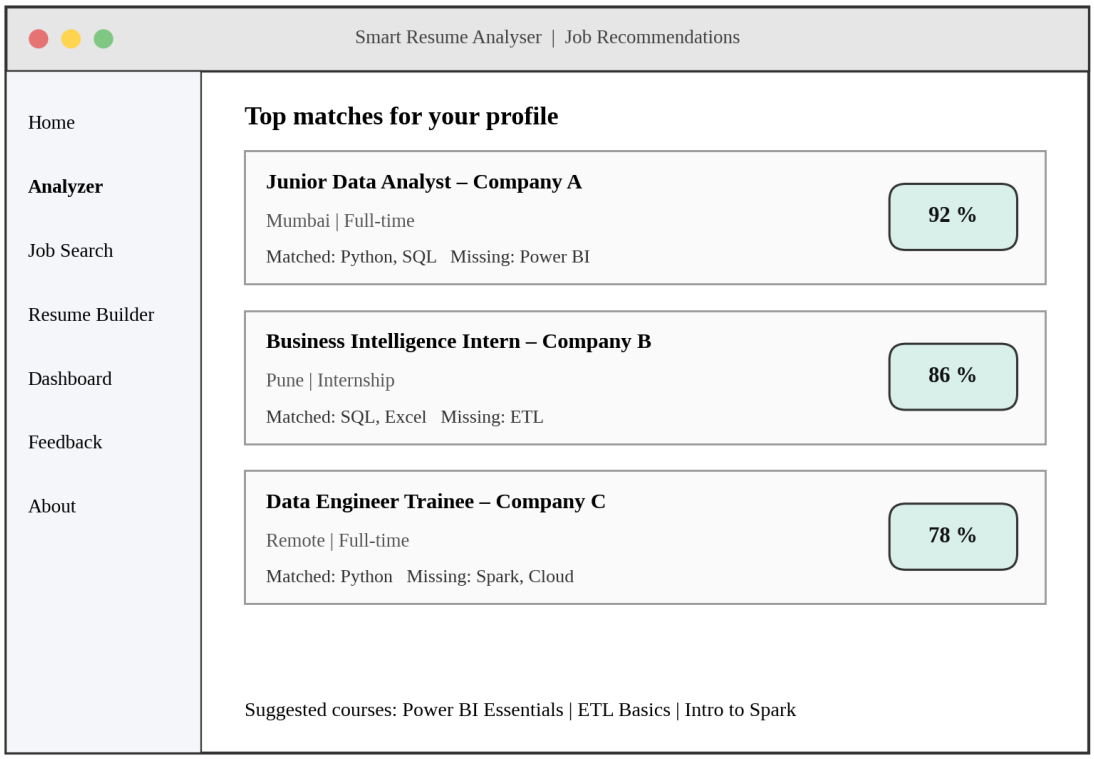


Fig. 3.9: GUI 03 – Job recommendations with explanations

3.5.8 Dataset preparation and ethical handling

Two datasets will be prepared. The first is a set of about 100 resumes collected from volunteers who give written, informed consent; names, contact details and addresses are removed before storage, and each file is given a random identifier. The second is a set of about 50 job descriptions covering software, data and business roles, taken from public postings or open datasets whose terms permit research use. Two annotators will independently label skills, education and experience in the resumes and rate the relevance of resume–job pairs on a three-point scale; disagreements will be resolved by discussion and agreement will be reported. The datasets will be used only for evaluation of this project and will not be published without further consent.

Job-posting data will be obtained only from sources whose terms of use allow it, such as public APIs, open datasets or employer-provided feeds. Automated scraping of sites that prohibit it will be avoided; this is a deliberate departure from the scraping-based job search described for the reference project [16].

3.5.9 Security and deployment

The application will be packaged as a Docker container and deployed on a cloud server behind HTTPS. Passwords will be stored as salted bcrypt hashes, sessions will expire after inactivity, and administrator functions will be limited by role. Uploaded files will be validated for type and size, parsed in a restricted environment and never executed. API keys and database credentials will be loaded from environment

variables and excluded from the code repository, and no default administrator password will be shipped. Application logs will record events and error codes but not resume text. A GitHub Actions workflow will run unit tests and basic dependency checks on every commit.

3.5.10 Testing and evaluation procedure

Evaluation will use a set of about 100 consenting, anonymised resumes and 50 job descriptions assembled across several roles. Parsing quality will be measured by field-level precision, recall and F1 against manual annotation. Score validity will be checked by comparing system scores with ratings from two or more experienced evaluators using rank correlation. Recommendation quality will be measured with Precision@K, Mean Reciprocal Rank (MRR) and NDCG against human relevance judgements. Non-functional tests will measure response time, concurrent-user behaviour and security checks. Usability will be assessed with the System Usability Scale (SUS) questionnaire [29].

3.6 Components / Materials

The project is software-only. Table 3.2 lists the tools, libraries and equipment required.

Table 3.2: Software components and specifications

Component	Specification	Quantity
Programming language	Python 3.11 or later	1
Web framework / UI	Streamlit 1.x (HTML/CSS for custom styling)	1
NLP library	spaCy (en_core_web_sm) for tokenisation, NER and matching; NLTK for stop-words and stemming	1 each
Text extraction	pdfplumber / PyPDF2 for PDF; python-docx for DOCX; optional Tesseract OCR	1 set
Embedding model	sentence-transformers “all-MiniLM-L6-v2” (384-dimensional vectors)	1
ML / statistics	scikit-learn (TfidfVectorizer, cosine similarity), NumPy, pandas	1 set
LLM service	Google Gemini API (key stored in environment variable)	1 key
Database	SQLite 3 for development; PostgreSQL optional for deployment	1
Visualisation / reports	Plotly for charts; ReportLab/FPDF for PDF report; openpyxl for Excel export	1 set
Security	bcrypt for password hashing; python-dotenv for secrets	1 set
Testing and DevOps	pytest, Git/GitHub, Docker, GitHub Actions	1 set
Development hardware	Laptop with 8 GB RAM or more, 4-core CPU, 20 GB free storage	4

CHAPTER 4: PROJECT PLANNING

4.1 Work Breakdown Structure

The project is divided into nine work packages spread over one semester of about 19 weeks (Table 4.1). Each package has a defined deliverable so that progress can be reviewed with the guide at regular intervals.

Table 4.1: Work breakdown structure and planning table

WP	Work package	Key deliverable	Weeks	Duration
1	Requirement analysis and literature survey	SRS draft, literature table, feature list	1–3	3 wk
2	System design	DFD, UML, ER diagram, UI wireframes	3–5	3 wk
3	Resume parser module	Text extraction, sectioning, skill extraction	5–8	4 wk
4	Scoring engine	ATS rules, TF-IDF and SBERT scoring	7–10	4 wk
5	Job recommender and skill-gap module	Embedding store, ranking, explanations	10–13	4 wk
6	Web interface and resume builder	Streamlit pages, templates, PDF export	11–14	4 wk
7	Admin panel, security and privacy	Authentication, hashing, PII masking, retention	13–15	3 wk
8	Integration and testing	Test report, bug fixes	15–17	3 wk
9	Evaluation and documentation	Evaluation results, final report, presentation	17–18	2 wk

4.2 Gantt Chart

Fig. 4.1 presents the semester plan. Packages 3 to 6 overlap deliberately so that the four team members can work in parallel, with weekly integration to detect interface problems early.

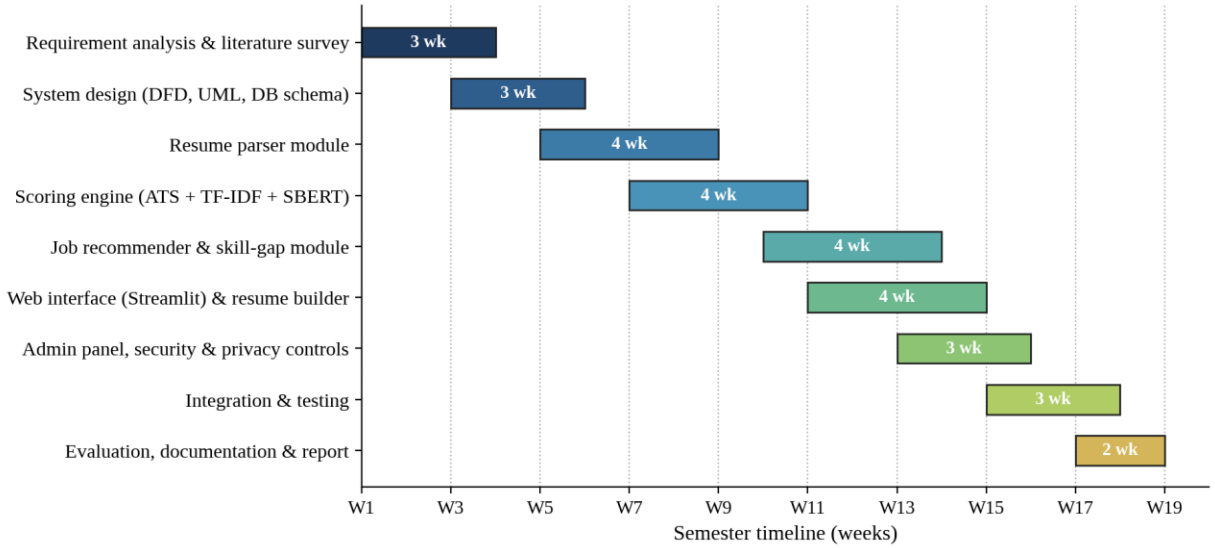


Fig. 4.1: Gantt chart for semester planning

4.3 Roles and Responsibilities

The proposed allocation below ensures that every member contributes to design, coding and testing. All members will take part in documentation and in peer code review.

Table 4.2: Roles and responsibilities (proposed)

Student	Responsibility
Kunal Bhandarkar (EU1237030)	Team coordination; scoring engine (ATS rules, TF-IDF, semantic score); integration and deployment
Rishi Jadhav (EU1237028)	Resume parser; skill extraction and normalisation; evaluation dataset and annotation
Nishit Manjrekar (EU1237038)	Streamlit user interface; resume builder and PDF report; database design and admin dashboard
Harsh Patil (EU2237012)	Job recommender and skill-gap module; LLM integration and privacy controls; testing and documentation

4.4 Risk Assessment

Table 4.3: Risk assessment

Risk	Impact	Mitigation
Resume layouts that parsers cannot read (columns, tables, images)	High	Use two extraction libraries with fallback; optional OCR; show a clear “unreadable section” warning
LLM API rate limits, cost or downtime	Medium	Cache results; fall back to rule-based suggestions; support more than one provider
Job data unavailable or restricted by a source’s terms of use	High	Use licensed/public APIs and open datasets; avoid scraping sites that prohibit it; keep a seed dataset
Privacy breach or misuse of personal data	High	PII masking, encryption in transit, consent, retention limit, access control, no logging of resume text
Biased or misleading scores	High	Exclude name, gender, age and photo from scoring; test on diverse samples; show component scores
Inaccurate skill extraction	Medium	Maintain alias table; manual review of errors; iterate with annotated sample
Schedule slippage	Medium	Two-week sprints, weekly stand-ups, buffer in weeks 17–18
Team member unavailability	Low	Pair programming; shared repository and documentation

4.5 Milestones and Monitoring

Progress will be monitored through weekly meetings of the team, fortnightly reviews with the guide and a task board in the GitHub repository. Six milestones (Table 4.4) mark the points at which a working deliverable is demonstrated.

Table 4.4: Project milestones

No.	Milestone	Evidence of completion	Week
M1	Requirements frozen	Approved SRS and literature table	3
M2	Design complete	DFD, UML, ER diagram and wireframes reviewed by guide	5
M3	Alpha release	Parser and scoring engine working on sample resumes	10
M4	Beta release	Recommender, UI and resume builder integrated	14
M5	Testing complete	Test report; security and privacy checks passed	17
M6	Final submission	Evaluation results, report and demonstration	19

4.6 Version Control and Quality Practices

- **Branching:** a protected main branch, a develop branch for integration and short-lived feature branches named after tasks.
- **Code review:** every pull request is reviewed by at least one other team member before merging.
- **Testing:** unit tests for the parser, scorer and recommender run automatically on each commit.
- **Issue tracking:** user stories and bugs are recorded as GitHub issues and linked to milestones.
- **Documentation:** a README, setup guide and short design notes are updated with every milestone.
- **Licensing:** third-party libraries are used under their open-source licences; the reference project is cited and no code is copied without attribution, in line with its MIT licence [16].

CHAPTER 5: RESOURCES & FEASIBILITY

5.1 Technical Feasibility

All major building blocks already exist as mature open-source software: spaCy and NLTK for language processing, scikit-learn for TF-IDF and similarity, and sentence-transformers for embeddings. The chosen embedding model is small (about 384-dimensional vectors) and runs on a CPU, so no GPU is required. Streamlit allows a data-driven web interface to be written in Python alone, which suits the team's skills and shortens development time. The reference open-source project demonstrates that a similar application can be built and hosted on a free community platform [16], [22].

The main technical risks are parsing accuracy for complex layouts and dependence on third-party APIs. Both are mitigated by fallback methods and caching, as described in Table 4.3. The team has prior coursework in Python, databases, machine learning and web development, and the guide's expertise will be used for architecture review. The project is therefore judged technically feasible.

5.2 Economic Feasibility

The project requires no specialised hardware. Development uses the team's own laptops; software is free and open source; and cloud hosting, a domain name and limited paid API usage are the only cash costs. The total estimated budget is about ₹11,600 (Table 5.1), which is modest for a final-year project. If the portal is later offered to institutions, hosting and LLM costs scale with usage and can be recovered through a subscription or a placement-cell licence.

5.3 Operational Feasibility

The system is operated through a browser and needs no installation by the user. The interface is designed around three tasks, namely upload, read results and view jobs, which can be learned within minutes. Administrators need only basic training to view statistics and manage users. Maintenance consists of updating the job store, refreshing the skill dictionary and rotating API keys, all of which can be scripted. Feedback forms will be used to gather user suggestions after each analysis.

5.4 Budget Estimation

Table 5.1: Estimated budget (INR)

Item	Quantity	Unit cost (₹)	Cost (₹)
Cloud server / hosting (2 vCPU, 4 GB RAM), 6 months	6	700	4,200
Domain name (1 year)	1	800	800
LLM API usage beyond free tier (testing)	1	2,000	2,000
Job-listing API or dataset (paid tier if required)	1	1,500	1,500
Documentation: printing and binding	4	400	1,600
Contingency	1	1,500	1,500
Total			**11,600**

The figures are planning estimates and will be revised against actual prices at the time of purchase.

5.5 Bill of Materials (BOM)

Table 5.2: Bill of materials – hardware and software

Item	Specification / licence	Qty	Cost (₹)
Development laptops	8 GB RAM or more, 4-core CPU, Windows/Linux (owned by team)	4	Nil
Python and libraries	Python 3.11; spaCy, NLTK, scikit-learn, sentence-transformers, pandas (open source)	1 set	Nil
Web framework	Streamlit (Apache-2.0)	1	Nil
Database	SQLite / PostgreSQL (open source)	1	Nil
Version control and CI	Git, GitHub (free plan), GitHub Actions	1	Nil
LLM service	Google Gemini API key (free tier, paid if needed)	1	2,000
Hosting and domain	Cloud VPS 2 vCPU/4 GB; domain with HTTPS certificate	1	5,000
Job data	Public API or open dataset	1	1,500

CHAPTER 6: EXPECTED RESULTS

This chapter describes what the completed system is expected to achieve. The values in Table 6.1 are design targets set before implementation; they are not measured results and will be verified during the evaluation phase.

6.1 Expected Prototype

A deployed web application with user registration, resume upload, analysis dashboard, job recommendations, resume builder, PDF report download, feedback form and administrator panel. The prototype will accept PDF and DOCX resumes and produce an overall score with its five components, a list of matched and missing keywords, LLM-written suggestions and at least five ranked job recommendations with explanations.

6.2 Expected Accuracy

The parser is expected to extract skills, education and experience with a field-level F1 score of at least 0.85 on well-formatted resumes, with lower performance anticipated on heavily designed layouts. The scoring engine is expected to show a positive rank correlation of at least 0.70 with ratings assigned by experienced human evaluators. For recommendations, at least 70 % of the top-five results are expected to be judged relevant by evaluators.

6.3 Expected Efficiency and Performance

Analysis excluding the LLM call is expected to complete in under 10 seconds for a two-page resume on a standard cloud instance, and the complete flow including LLM feedback in under 20 seconds. Job embeddings will be pre-computed, so ranking a store of several thousand postings is expected to take well under a second. The system is expected to serve about 20 concurrent users on the planned server.

6.4 Expected Experimental Results

The experiments planned in Section 3.5.10 are expected to show that the hybrid score aligns better with human judgement than keyword overlap alone, and that adding semantic similarity improves recommendation quality compared with a TF-IDF-only baseline. These hypotheses will be tested through an ablation study in which each component of the score is removed in turn. A usability study with at least 20 students is expected to yield a System Usability Scale score of 70 or above, which is generally regarded as acceptable usability [29].

Table 6.1: Expected performance targets

Metric	Target	Evaluation method
Field-level extraction F1	≥ 0.85 (skills, education, experience)	Manual annotation of 100 resumes
Score–expert rank correlation	≥ 0.70 (Spearman)	Ratings by two or more evaluators
Precision@5 of recommendations	≥ 0.70	Human relevance judgements
NDCG@10	≥ 0.75	Graded relevance on 50 job descriptions
Analysis time (without LLM)	≤ 10 s per resume	Timed tests on server
End-to-end time (with LLM)	≤ 20 s per resume	Timed tests on server
Concurrent users	≥ 20 without errors	Load test script
Usability (SUS)	≥ 70	Questionnaire, 20 or more users

6.5 Baselines and Ablation Plan

To show that each technique contributes, four system configurations will be compared on the same evaluation data (Table 6.2). The hybrid system is expected to perform best, and the differences between configurations will indicate the value of semantic matching and of the explainability layer.

Table 6.2: Planned configurations for ablation study

ID	Configuration	Purpose
B1	Keyword overlap only	Simple baseline similar to basic resume checkers
B2	TF-IDF weighted matching	Tests the benefit of term weighting [3]
B3	Sentence-BERT similarity only	Tests the benefit of semantic matching [2]
H1	Hybrid: rules + TF-IDF + SBERT + skill overlap	Proposed system; full scoring and recommendation pipeline

6.6 Traceability of Objectives to Expected Evidence

Table 6.3: Objectives and the evidence that will show they are met

Objective (Section 1.4)	Expected evidence
1. Resume-parsing pipeline	Extraction F1 on annotated resumes; list of supported formats
2. Hybrid scoring engine	Score–expert correlation; ablation results (Table 6.2)
3. Explainable job recommendation	Precision@5, NDCG@10; sample cards with matched/missing skills
4. LLM feedback with masking	Prompt audit showing no personal identifiers; user ratings of usefulness
5. Secure web portal	Working modules; security checklist and test cases (Appendix B)
6. Testing and documentation	Test report, timing results, SUS score and final report

CHAPTER 7: SUSTAINABILITY & IMPACT

7.1 Industrial Relevance

Recruitment technology is a large and growing segment of the software industry, and candidate-facing tools are increasingly adopted by universities and training providers. The proposed portal is relevant to campus placement cells, career-services providers, ed-tech companies and HR-technology start-ups. Its modular architecture allows the resume-analysis component to be used independently as an API, or integrated with an institution's learning-management and placement systems. The project also trains the team in tools widely used in industry: Python, NLP, transformers, REST-style services, containers and continuous integration.

7.2 Societal Impact

By giving every student the same quality of feedback regardless of access to mentors, the system can help reduce information gaps between well-connected and first-generation job seekers. Explanations of why a resume scored low, and which skills are missing, convert a rejection into a learning opportunity. At the same time, automated hiring tools have been criticised for reproducing bias; a widely reported case involved a recruiting tool that was withdrawn after it was found to disadvantage women [25], and reviews of hiring tools note weak evidence of validity [14], [15]. For this reason the design excludes names, gender and photographs from scoring, shows its reasoning, treats LLM advice as advisory, and does not make hiring decisions.

7.3 Environmental Impact

The project is software-only and produces no manufacturing waste. Its main environmental footprint is the electricity used by development machines, cloud servers and LLM inference. This footprint is kept small by choosing a compact embedding model that runs on a CPU, caching embeddings and LLM responses, calling the LLM only when the user requests suggestions, and hosting on a shared instance with auto-sleep when idle. A paperless job-application workflow, in which resumes are prepared and submitted digitally, also avoids printing.

7.4 SDG Mapping

The project contributes to the following United Nations Sustainable Development Goals [24]:

Table 7.1: Mapping of the project to the SDGs

	Goal	Contribution of the project
☑	SDG 4: Quality Education	Skill-gap analysis and course recommendations support lifelong learning and employability
☑	SDG 8: Decent Work and Economic Growth	Improves access to suitable jobs and reduces mismatch between skills and openings
☑	SDG 9: Industry, Innovation and Infrastructure	Applies AI/NLP in a deployable digital service built on open-source tools
☑	SDG 10: Reduced Inequalities	Free, consistent feedback for all students; bias-aware design
☑	SDG 16: Peace, Justice and Strong Institutions	Transparent, privacy-respecting use of personal data and accountable AI

7.5 Future Scope

- Support for Hindi, Marathi and other Indian languages, and OCR for scanned resumes.
- Interview preparation: question generation and answer feedback based on the resume and job description.
- Employer-side module with fairness audits, such as comparing score distributions across groups.
- Learning-to-rank using click and save data; retrieval-augmented generation grounded in a verified skills and courses knowledge base [5].
- Integration with college placement portals, mobile application, and fine-tuning of a domain-specific embedding model on resume–job pairs.

CHAPTER 8: CONCLUSION

Problem

Job seekers, especially students, do not know how automated screening systems read their resumes, which of their skills are visible to such systems, or which openings fit their profile. Existing tools either score resumes superficially or recommend jobs without explanation, and few address privacy and bias.

Proposed solution

This synopsis proposes an AI Resume Analyser and Job Recommendation Portal that combines resume parsing, hybrid scoring (ATS rules, TF-IDF and Sentence-BERT), LLM-generated suggestions and explainable, content-based job recommendation in one web application. The design emphasises transparency, privacy-by-design, secure authentication and reproducibility with open-source tools. The architecture, algorithms, data model, interface designs, project plan, budget and risk analysis have been described, and the feasibility of the project has been established on technical, economic and operational grounds.

Expected contribution

The completed project is expected to deliver a deployable prototype, an evaluation dataset and a documented methodology that other students and institutions can reuse. It will give candidates actionable feedback and job suggestions, help placement cells prepare batches of students, and demonstrate how modern NLP can be applied responsibly to recruitment. It will also give the team practical experience across the software life cycle, from requirements to deployment and evaluation.

REFERENCES

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, Hong Kong, 2019.

[3] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, 1988.

[4] V. Bhatia, P. Rawat, A. Kumar, and R. R. Shah, "End-to-end resume parsing and finding candidates for a job description using BERT," arXiv:1910.03089, 2019.

[5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[6] S. Maheshwary and H. Misra, "Matching resumes to jobs via deep Siamese network," in Companion Proc. The Web Conference (WWW), 2018.

[7] T. B. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[8] C. Qin et al., "Enhancing person-job fit for talent recruitment: An ability-aware neural network approach," in Proc. ACM SIGIR, 2018.

[9] C. Zhu et al., "Person-job fit: Adapting the right talent for the right job with joint representation learning," ACM Trans. Management Information Systems, vol. 9, no. 3, 2018.

[10] M. Zhao et al., "SKILL: A system for skill identification and normalization," in Proc. AAAI Conf. Artificial Intelligence, 2015.

[11] K. Kenthapadi, B. Le, and G. Venkataraman, "Personalized job recommendation system at LinkedIn: Practical challenges and lessons learned," in Proc. ACM RecSys, 2017.

[12] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based recommender systems: State of the art and trends," in Recommender Systems Handbook, F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds. Boston, MA, USA: Springer, 2011.

[13] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 2nd ed. New York, NY, USA: Springer, 2015.

[14] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in Proc. ACM FAT*, 2020.

[15] M. Bogen and A. Rieke, "Help wanted: An examination of hiring algorithms, equity, and bias," Upturn, 2018.

[16] H. Patel (Hunterdii), "Smart AI Resume Analyzer," GitHub repository, MIT License. [Online]. Available: https://github.com/Hunterdii/Smart-AI-Resume-Analyzer (accessed Oct. 2026).

[17] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Machine Learning Research, vol. 12, 2011.

[18] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.

[19] Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, ISO/IEC 25010:2011, 2011.

[20] Government of India, "The Digital Personal Data Protection Act, 2023," Ministry of Law and Justice, 2023.

[21] M. Honnibal and I. Montani, "spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing," 2017. [Online]. Available: https://spacy.io

[22] Streamlit Inc., “Streamlit documentation.” [Online]. Available: https://docs.streamlit.io

[23] A. Vaswani et al., “Attention is all you need,” in Advances in Neural Information Processing Systems (NIPS), 2017.

[24] United Nations General Assembly, “Transforming our world: The 2030 Agenda for Sustainable Development,” A/RES/70/1, 2015.

[25] J. Dastin, “Amazon scraps secret AI recruiting tool that showed bias against women,” Reuters, Oct. 10, 2018.

[26] Gemini Team, Google, “Gemini: A family of highly capable multimodal models,” arXiv:2312.11805, 2023.

[27] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. Sebastopol, CA, USA: O’Reilly Media, 2009.

[28] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

[29] J. Brooke, “SUS: A ‘quick and dirty’ usability scale,” in Usability Evaluation in Industry. London, U.K.: Taylor & Francis, 1996.

APPENDICES

Appendix A: Requirement Summary

Functional requirements

- FR1: The system shall allow a user to register, log in and log out securely.
- FR2: The system shall accept PDF and DOCX resumes up to 5 MB and reject other formats.
- FR3: The system shall extract and display skills, education, experience and projects.
- FR4: The system shall compute and display an overall score and five component scores.
- FR5: The system shall compare a resume with a pasted job description and list matched and missing keywords.
- FR6: The system shall recommend at least five jobs with matched skills, missing skills and a similarity score.
- FR7: The system shall generate LLM-based suggestions after masking personal identifiers.
- FR8: The system shall let the user build a resume from templates and download it as PDF.
- FR9: The administrator shall view user statistics and feedback and manage accounts.

Non-functional requirements

- NFR1 (Performance): analysis without LLM within 10 seconds for a two-page resume.
- NFR2 (Security): passwords hashed with salt; secrets kept outside source code; HTTPS in deployment.
- NFR3 (Privacy): consent before upload; deletion on request; automatic deletion after the retention period.
- NFR4 (Usability): primary tasks reachable within three clicks; SUS score of 70 or above.
- NFR5 (Maintainability): modular code, documented interfaces and automated tests for core modules.

Appendix B: Sample Test Cases

Table A.1: Sample test cases

ID	Test	Expected result	Module
TC1	Upload a valid PDF resume	Text extracted; sections detected	Parser
TC2	Upload an unsupported file (.exe)	Error message; upload rejected	Parser / UI
TC3	Resume with “JS” and “JavaScript”	Counted as one skill	Skill normaliser
TC4	Paste job description with missing skills	Missing skills listed correctly	Scoring engine
TC5	Request recommendations	Top-5 jobs with explanations	Recommender
TC6	Inspect LLM prompt	No name, e-mail or phone present	Privacy module
TC7	Login with wrong password	Access denied; no hint about which field is wrong	Authentication
TC8	Delete my data	Resume and analyses removed from database	Data layer

Appendix C: Survey Questionnaire (Usability and Usefulness)

Participants will rate each statement from 1 (strongly disagree) to 5 (strongly agree). Items 1 to 10 follow the System Usability Scale [29]; items 11 to 14 are project-specific.

1. I think that I would like to use this system frequently.
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think I would need technical support to use this system.
5. I found the various functions in this system well integrated.
6. I thought there was too much inconsistency in this system.
7. I imagine most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.
10. I needed to learn a lot of things before I could get going with this system.
11. The resume suggestions were clear and actionable.
12. The recommended jobs matched my skills and interests.
13. The explanation of missing skills helped me decide what to learn next.
14. I am comfortable with how my personal data was handled.

Appendix D: Sample LLM Prompt Template

The prompt is built from computed findings and the masked profile; it does not contain the candidate’s name, e-mail, phone number or address.

```
SYSTEM: You are a career coach. Use only the data given. Reply as JSON.
ROLE:   {target_role}
SCORES: ats={F}, keywords={K}, semantic={M}, completeness={C}, quality={Q}
MISSING_KEYWORDS: {k1, k2, k3}
PROFILE (masked): {summary, skills, experience bullets, projects}
TASK:   Return {strengths[], weaknesses[], rewritten_bullets[3], next_steps[]}
RULES:  Do not invent employers, degrees or numbers. Keep each item under 30 words.
```

Appendix E: Example Skill-Alias Table

Table A.2: Examples of skill normalisation

Surface form in resume	Canonical skill	Category
JS, Javascript, ECMAScript	JavaScript	Programming language
ML, machine-learning	Machine Learning	Data / AI
Postgres, PostgreSQL	PostgreSQL	Database
Power-BI, PowerBI	Power BI	Analytics tool
NLP, natural language processing	Natural Language Processing	Data / AI
k8s, Kubernetes	Kubernetes	DevOps

Appendix F: Project Canvas

Problem Students do not know how ATS reads their resumes or which jobs fit them	**Solution** Explainable resume scoring, LLM feedback and job recommendation in one portal	**Users** Students, graduates, placement cells
Key technologies Python, spaCy, scikit-learn, Sentence-BERT, Streamlit, SQLite, Gemini API	**Value proposition** Actionable, private, explainable career guidance at low cost	**Impact** Better employability; reduced skill mismatch; fairer screening
Costs Hosting, API usage, documentation (about ₹11,600)	**Revenue possibilities** Institutional licences; premium reports	**Risks** Parsing errors, API limits, data privacy, bias